

ENGEE319-26B Laboratory Session 1

Monitoring Equipment via WiFi

Activities: Write a program on the Raspberry Pi which will connect to a WiFi enabled smart plug, and gather power usage data on appliances. This data will be stored in csv files, displayed using spreadsheet software.

Introduction

The TP-Link HS110 smart plug is a typical smart home gadget. It provides two functions: remote on/off; and remote power measurement. It makes it possible to incorporate simple devices, those which can be safely controlled just by turning their power on and off (like lights, heaters, fans, etc.), into a smart home.

The original idea of IOT – the Internet of Things – was that individual devices would connect to the internet and thus be controllable from anywhere and able to provide information to anywhere. The equipment we have used thus far in ENGEE319 has mostly had more limited connectivity. The thermistor sensors we used had to be wired to Raspberry Pi computers to function. The MicroBit devices were capable of connecting over a limited distance using Bluetooth radio. In each case however, the Raspberry Pi was capable of communicating to the internet if required. We simply didn't take advantage of that capability.



The TP-Link smart plug connects via WiFi, which limits its immediate range to an area about the size of a house. However, the expected usage scenario is having it connect to home WiFi and through the home's internet connection, on to the outside world. In this sense WiFi isn't a limitation on connection; it is just a bridge to the wider internet and global communication. As a device on the internet, it has (in principal) an IP address which is globally accessible. In practice the address is an IPV4 address. There are something less than 4 billion possible addresses worldwide (all allocated: [IPv4 address exhaustion - Wikipedia](#)). There were 26.66 billion IOT devices in the world in 2019. Estimates reached 35 to 46 billion by 2021. IPV6 addresses can accommodate this number comfortably, but most smart home devices use IPV4. So, we can't make all IOT devices directly accessible on the internet very easily. In fact, we don't really want all of our devices directly accessible from anywhere – it might not be all that great if anyone anywhere could turn our heaters on and off. So a modern smart home has a single (usually IPV4) internet address and hides all the devices inside behind a kind of firewall – a NAT interface (NAT stands for Network Address Translation). This is set up so that devices can initiate connection to sites outside, but cannot be directly contacted from the outside unless special entries are programmed into the firewall – not too hard to do – but not something that we need to do most of the time. Note that the same limitations apply to all devices in our homes – desktop and laptop computers, phones using WiFi, etc.

Where does this leave us: The TP-Link in a normal household (or in the lab, connecting via the waikato_iot WiFi network), can talk to the world. The world can reply, but not initiate connections. This is the modern idea of IOT, clusters of devices in local networks, all able to send data to and request information from the outside world.

TP-Link embodies the IOT idea – internet connected devices; but with limitations on access.

Task 1: Using a fully IOT enabled device – the TP-Link HS110 smart plug

You can use the Smart Plug from your phone – in fact this is the manufacturer's main use case. The app they supply is called 'Kasa'.

1. Connect your mobile phone to the waikato_iot WiFi network. (password on whiteboard). *For the first few steps, this won't matter, but in order to connect to the HS110 device you will need to have it and your phone on the same wireless internet network.*
2. Install the 'Kasa for Mobile' app. There are subtle variations on title – don't know if that matters – but for safety, make sure you get the exact one. (It is the one named on the instruction leaflet that comes with the TP-Link HS110 device.)
3. The app will expect you to create an account. We can discuss pros and cons of having accounts with internet based companies, but for the purposes of this course I have created a single account that we can all use. This will require some care in not interfering with each other's devices, but is probably preferable to creating multiple accounts.

The account to use is registered
with my email: coms0108@cs.waikato.ac.nz
and has password: engee319

4. Log the Kasa app in to the account. You may see devices already present, either from other teams, or ones that I have been experimenting with.
5. I will probably have given you a TP-Link HS110 which is unused, but it is preferable anyway to factory reset it before you start. To do that, plug it in, turn on the power (to it), and hold down its button (top edge, above the power LED) for 5-10 seconds, until the signal light starts flashing green and orange, then again until it blinks orange rapidly.
6. Register your TP-Link HS110 with the Kasa app, and connect it to the waikato_iot network. The app organizes devices by room – bedroom, dining room etc. You can make a room for your team, and minimize confusion with other teams' devices. Start by pressing '+' to the top right. If you have difficulty, use the quick start guide referenced below, or ask for assistance. You will also find it helpful to choose a memorable name for your device (mine is called 'Pluggy'). You may be asked to update the software in the plug. If so, agree and let it happen.

Here is the manufacturer's description of the HS110 device.

<https://www.tp-link.com/en/home-networking/smart-plug/hs110>

Here is the quick start guide for the device.

[1910011762_HS100HS110_V1_User_Guide.pdf \(tp-link.com\)](#)

7. Using the Kasa app: Work out how to turn the plug on and off. You can use the desk lamp as a load.
8. Work out how to get energy usage readings from the device. What information can you get?

9. Discussion points: The manufacturer states that you can control your smart plug from anywhere! To establish connection and register the device it was necessary to be on the same WiFi network. Once connection is established however, you may be able to log in with your phone from anywhere on the Internet, and access the switch (so long as it is plugged in, powered and connected). How is this possible? How secure is this access? How can you test that this is happening? Does the Kasa app give you any control of the process? Note that this implies that you are not accessing the device directly. In fact you are accessing it through a company managed server – ours is the Asia/Pacific one. Real IOT! Be prepared to answer or discuss these questions.

Task 2: Accessing the device in software (using Python)

It does not appear to have been the intention of the manufacturer to allow general software access to the device – i.e. they didn't provide an API. They do have arrangements with other companies to allow access – for example you can connect a TP-Link to Amazon's Alexa system and get verbal control in a household setting.

The following article is a review of the TP-Link by an internet security group.

<https://www.softscheck.com/en/reverse-engineering-tp-link-hs110/>

The article shows that the device can be accessed fairly easily, but didn't succumb to a takeover attack; so although there are security issues, it is middling-well secured. Note that security is an ongoing issue in the IOT space. The idea of having many little electronic helpers in our homes and communities is attractive. The idea of having malicious people able to alter their functionality is not attractive.

The outcome of the security group's work has been the creation of publically available, but unofficial, API's in Node.js and Python which allow us to control and access information from TPLink devices. One is called 'tplink-cloud-api'. It was used in previous versions of this Lab exercise. However it makes use of now deprecated features in web access and security libraries. These can be disabled, but rolling back what are effectively security patches is not good engineering practice. Instead I have updated the Lab exercise to use a different library: python-kasa

10. Create a directory and a Python environment for this Lab. Activate the environment and install python-kasa as follows. When you program using this library, you will also need a library called asyncio, so install that as well.

```
python -m venv pythonenv
```

```
source pythonenv/bin/activate
```

```
pip install python-kasa
```

```
pip install asyncio
```

The 'python-kasa' library has both a programming library (API), and a command line program (called 'kasa') which you can run directly. When you install a module into a Python environment using 'pip', programs are put into the python environment 'bin' directory. Activating an environment adds the 'bin' directory to your execution path.

11. The 'kasa' program has a number of command line options for interrogating and controlling devices. 'kasa --help' will list its options. If there are no commands, 'kasa' will explore the immediate local network for devices and list their current status. Try:

Kasa

12. What can you learn from the output of the program?

An interesting feature of the output is the 'host' IP address for each device. This is the local internet address for the device on the waikato_iot wireless network. Things to note:

The kasa program has not asked for credentials to access a Kasa account

It does have access to waikato_iot WiFi because it is running on the Raspberry Pi, which is logged in to waikato_iot.

It knows about IP addresses.

From this (and from the python-kasa documentation) we can conclude that this library and program do not access devices via TP-Link's server. Instead they access the device directly on local WiFi

13. Note the host IP address of your switch. Using that IP address, try the following

```
kasa --host xxx.xxx.xxx.xxx
```

What happens? Work out how to turn the switch on and off. If you plug in a load, check to see what power is being used. Be prepared to explain.

The python-kasa project is documented at

<https://python-kasa.readthedocs.io/en/latest/index.html>

The API (library to call from Python programs) is accessed from the 'Library usage' link

<https://python-kasa.readthedocs.io/en/latest/library.html>

Following sections give you a programs to try. They illustrate operation of python-kasa. Note the declaration of the function 'main' to be 'async' and the use of 'await' whenever functions which might take some time to complete are called. The practical lecture discusses some of the ideas of 'asynchronous' programming in Python.

14. This program uses the python-kasa library to discover devices. What results do you get?

plug01.py

```
from kasa import Discover import
asyncio
    async def
main():
    devs = await Discover.discover()
    print('Discovered ', len(devs), ' devices')
for dev in devs:
    print(dev)

asyncio.run(main())
```

15. Once the IP address of a device is known (it will be quite stable once set), it is possible to just access that device. Don't forget to change the IP address to the one your device is reporting. What does this return? You can use 'dev.alias' to print the 'name of your device.

plug02.py

```
from kasa import Discover import
asyncio
    async def
main():
    dev = await Discover.discover_single('192.168.1.4')
    print(dev)

asyncio.run(main())
```

16. Try the following to turn your plug on and off (with some load plugged in)

plug03.py

```
from kasa import Discover import
asyncio
    async def
main():
    dev = await
Discover.discover_single('192.168.1.4')    await
dev.turn_on()    await asyncio.sleep(1)    await
dev.turn_off()

asyncio.run(main())
```

17. You can ask a device if it incorporates an 'emeter' – i.e. if it can report energy usage. If it does, you can access the energy reporting module, and request usage information.

plug04.py

```
from kasa import Discover import
asyncio
    async def
main():
    dev = await
Discover.discover_single('192.168.1.4')    await
dev.update()    if not dev.has_emeter:
    print('Device can not report energy usage')
return
    e_module = dev.modules.get("Energy")
    while
True:
        await asyncio.sleep(1)
    await dev.update()
        print(e_module.current, 'A @', e_module.voltage, 'V')

asyncio.run(main())
```

18. Extend the last program to have a Control-C handler, so that it will stop without a lot of error messages. Extend it again compute and report energy usage in Watts. Is it possible to report Watt Hours?
19. Try running the energy reporting program while you turn the switch on and off with the app on your phone, or with the power button (on the front, not top) of the switch itself.

Task 3: Recording data

To get typical use data for appliances we need to record their power usage for some time (seconds, minutes, hours?). A widely used and flexible storage mechanism for this sort of statistical data is the comma separated text file (csv file). Python has a built in library (csv) for helping to make such files. Here is the outline of a program to record samples from a device at 5 second intervals for about 100 seconds.

20. Build this into your energy monitoring program and try it. Have a look at the csv file it creates.

plug05.py – making a CSV file

```
import csv
import time
csv_file = open('lamp.csv',
mode='w') csv_writer =
csv.writer(csv_file)
csv_writer.writerow(['Time', 'Current', 'Voltage',
'Power'])
# Make a connection to the energy monitoring plug
# ...
for count in range(0,
20):
    print(count)

    # Obtain an energy reading (current, voltage,
power)    ...

    csv_writer.writerow([int(time.time()),
                        #...current,
                        #...voltage,
#...power])    time.sleep(5)

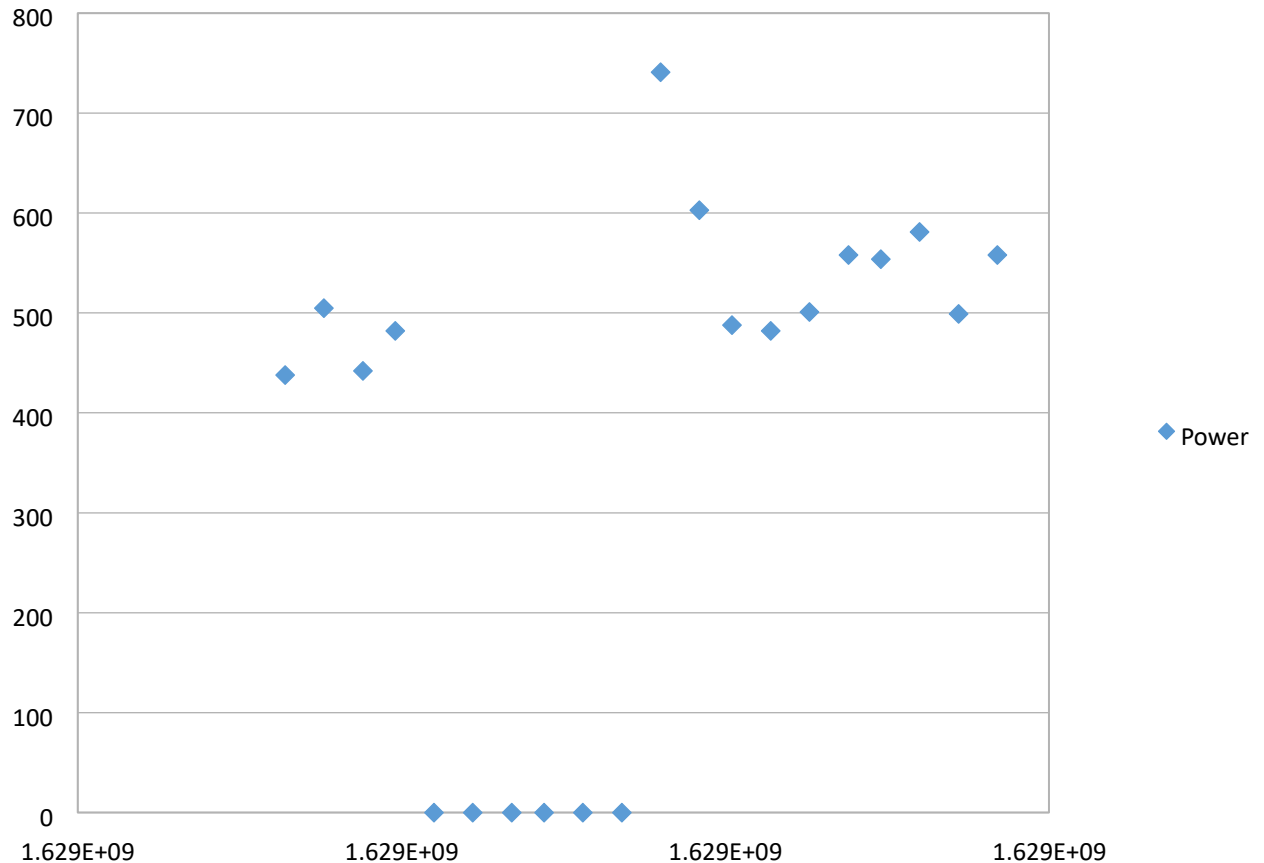
csv_file.close()
```

21. The program as it stands is quite inflexible – it runs for 100 seconds. Use a Control-C handler to close the file and stop the program. This should allow more flexible sampling. How accurate are the times? Does this matter?
22. Install Libre Office on your Raspberry Pi. Libre Office is an open source productivity application (word processor, spreadsheet, etc). From a command line type:

```
sudo apt install libreoffice
```

After it installs, you should be able to find it under 'Office' on the Raspberry Pi menu. Its spreadsheet is called 'LibreOffice Calc'.

23. Using Libre Office Calc, load a CSV file and use the chart system to make a graph of power usage against time. It might look something like the following. Can you improve the time scale and (perhaps) connect the dots?



24. You have appliances available: A desk lamp, a hair dryer, a toasted sandwich maker and an e-scooter. Create csv files showing usage patterns of these devices and make graphs. You will need to think about the way in which these devices are used, and decide on a reasonable sampling time for each. The csv files, or ones like them, will be used as part of the data for the mini project.